

Podprogramy a lambda funkce

Podprogram

- **podprogram** = oddělená část programu
- obsahuje kód, který můžeme opakovaně volat
- řeší jednu konkrétní část problému
- pomáhá rozdělit program na menší části
- tomu se říká **dekompozice**

Podprogram může být například:

- funkce
- procedura
- metoda

Proč používáme podprogramy

- kód se neopakuje zbytečně vícekrát
- program je přehlednější
- program se lépe testuje
- program se lépe opravuje
- složitý problém se rozdělí na menší části
- stejný kód lze použít na více místech

Funkce, procedura, metoda

Funkce

- **funkce** je podprogram, který vrací hodnotu
- má návratový typ
- obvykle používá klíčové slovo `return`
- může mít vstupní parametry

Příklad:

```
int Secti(int a, int b)
{
    return a + b;
}
```

Použití:

```
int vysledek = Secti(3, 5);  
Console.WriteLine(vysledek); // 8
```

Procedura

- **procedura** je podprogram, který nic nevrací
- jejím cílem je něco provést
- v C# se zapisuje návratovým typem `void`

Příklad:

```
void Pozdrav(string jmeno)  
{  
    Console.WriteLine("Ahoj " + jmeno);  
}
```

Použití:

```
Pozdrav("Jan");
```

Metoda

- **metoda** = funkce nebo procedura, která je součástí třídy
- používá se v objektově orientovaném programování
- metoda popisuje chování objektu

Příklad:

```
class Auto  
{  
    public void Jed()  
    {  
        Console.WriteLine("Auto jede.");  
    }  
}
```

Použití:

```
Auto auto = new Auto();  
auto.Jed();
```

Stavba funkce

Funkce se skládá hlavně z těchto částí:

- název
- parametry
- návratový typ
- tělo funkce
- návratová hodnota

Příklad:

```
int Vynasob(int a, int b)
{
    return a * b;
}
```

Vysvětlení:

- `int` = návratový typ
- `Vynasob` = název funkce
- `int a, int b` = parametry
- `{ ... }` = tělo funkce
- `return a * b` = návratová hodnota

Parametry a argumenty

Parametr

- **parametr** je proměnná uvedená v definici funkce
- říká, jaká data funkce očekává

Příklad:

```
int Secti(int a, int b)
{
    return a + b;
}
```

Argument

- **argument** je konkrétní hodnota předaná při volání funkce

Příklad:

```
int vysledek = Secti(3, 5);
```

- `3` a `5` jsou argumenty

Předávání parametrů

Předávání hodnotou

- funkce dostane kopii hodnoty
- změna uvnitř funkce neovlivní původní proměnnou
- běžné u hodnotových typů

Příklad:

```
void Zvys(int x)
{
    x++;
    Console.WriteLine("Uvnitř funkce: " + x);
}

int cislo = 5;
Zvys(cislo);

Console.WriteLine("Venku: " + cislo);
```

Výstup:

```
Uvnitř funkce: 6
Venku: 5
```

- uvnitř funkce se změnila jen kopie
- původní `cislo` zůstalo 5

Předávání referencí pomocí `ref`

- funkce pracuje přímo s původní proměnnou
- změna uvnitř funkce se projeví i venku
- proměnná musí být předem inicializovaná
- při volání i definici se píše `ref`

Příklad:

```
void Zvys(ref int x)
{
    x++;
}

int cislo = 5;
Zvys(ref cislo);
```

```
Console.WriteLine(cislo); // 6
```

Parametr out

- používá se pro vrácení hodnoty přes parametr
- funkce musí do `out` parametru přiřadit hodnotu
- proměnná nemusí být předem inicializovaná
- často se používá, když metoda vrací více informací

Příklad:

```
bool uspech = int.TryParse("123", out int cislo);

if (uspech)
{
    Console.WriteLine(cislo); // 123
}
else
{
    Console.WriteLine("Převod se nepovedl.");
}
```

Vysvětlení:

- `TryParse` vrací `bool`
- přes `out int cislo` vrací převedené číslo

Rozdíl mezi ref a out

Vlastnost	ref	out
Proměnná musí mít hodnotu před voláním	ano	ne
Funkce může hodnotu číst	ano	ne před přiřazením
Funkce musí hodnotu nastavit	ne nutně	ano
Použití	změna existující hodnoty	vrácení další hodnoty

Obor platnosti proměnné

- **obor platnosti** určuje, kde je proměnná dostupná
- anglicky **scope**
- proměnná existuje jen v určité části programu

- mimo svůj obor platnosti ji nelze použít

Lokální proměnná

- proměnná vytvořená uvnitř funkce nebo metody
- dostupná jen uvnitř této funkce nebo metody

Příklad:

```
void Test()
{
    int x = 10;
    Console.WriteLine(x);
}

// Console.WriteLine(x); // chyba
```

Blokový scope

- proměnná existuje jen uvnitř bloku
- blok je obvykle ohraničený složenými závorkami { }
- typicky:
 - if
 - for
 - while

Příklad:

```
if (true)
{
    int x = 10;
    Console.WriteLine(x);
}

// Console.WriteLine(x); // chyba
```

Proměnná v cyklu

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}

// Console.WriteLine(i); // chyba
```

- proměnná `i` existuje jen uvnitř cyklu `for`

Globální / členská proměnná

- v C# se běžně používá spíš pojem **členská proměnná** nebo **atribut třídy**
- je definovaná ve třídě
- mohou ji používat metody dané třídy
- dostupnost závisí na modifikátoru přístupu

Příklad:

```
class Student
{
    private string jmeno;

    public void NastavJmeno(string noveJmeno)
    {
        jmeno = noveJmeno;
    }

    public void VypisJmeno()
    {
        Console.WriteLine(jmeno);
    }
}
```

- `jmeno` je dostupné uvnitř třídy `Student`
- mimo třídu záleží na tom, jestli je `private`, `public`, apod.

Rekurze

Co je rekurze

- **rekurze** = funkce volá sama sebe
- používá se místo cyklu
- problém se rozkládá na menší podproblémy
- každé volání řeší menší část problému

Rekurzivní funkce musí mít

1. Ukončovací podmínku

- říká, kdy se má rekurze zastavit
- bez ní by funkce volala sama sebe donekonečna
- mohlo by dojít k chybě **StackOverflow**

2. Rekurzivní volání

- funkce zavolá sama sebe
- většinou s menší nebo jednodušší hodnotou

Příklad: faktoriál

Faktoriál čísla n :

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Rekurzivní řešení:

```
int Faktorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }

    return n * Faktorial(n - 1);
}
```

Použití:

```
Console.WriteLine(Faktorial(5)); // 120
```

Vysvětlení:

- `if (n <= 1)` = ukončovací podmínka
- `Faktorial(n - 1)` = rekurzivní volání

Iterace vs. rekurze

Vlastnost	Iterace	Rekurze
Princip	cyklus	funkce volá sama sebe
Ukončení	podmínka cyklu	ukončovací podmínka
Riziko	nekonečný cyklus	StackOverflow
Použití	běžné opakování	stromové struktury, faktoriál
Paměť	obvykle úspornější	každé volání zabírá místo na zásobníku

Iterační faktoriál


```
int FaktorialIterativne(int n)
{
    int vysledek = 1;

    for (int i = 2; i <= n; i++)
    {
        vysledek *= i;
    }

    return vysledek;
}
```

Anonymní funkce

- **anonymní funkce** = funkce bez vlastního jména
- často se ukládá do proměnné
- používá se hlavně tam, kde chceme funkci předat jako hodnotu
- v C# se často používá přes delegát nebo lambda výraz

Příklad anonymní funkce přes delegát:

```
Func<int, int> zvys = delegate (int x)
{
    return x + 1;
};

Console.WriteLine(zvys(5)); // 6
```

Lambda funkce

Co je lambda funkce

- **lambda funkce** = krátký zápis anonymní funkce
- nemá vlastní název
- často se používá jako argument jiné funkce
- v C# se zapisuje pomocí operátoru `=>`

Oběcný zápis:

```
parametr => výraz
```

Příklad:

```
x => x + 1
```

Lambda uložená do proměnné

```
Func<int, int> zvys = x => x + 1;

Console.WriteLine(zvys(5)); // 6
```

Vysvětlení:

- `Func<int, int>` znamená:
 - přijímá `int`
 - vrací `int`
- `x => x + 1` je lambda funkce

Lambda s více parametry

```
Func<int, int, int> secti = (a, b) => a + b;

Console.WriteLine(secti(3, 5)); // 8
```

Lambda s tělem

- pokud má lambda více příkazů, používají se složené závorky
- návratová hodnota se vrací přes `return`

Příklad:

```
Func<int, int> absolutniHodnota = x =>
{
    if (x < 0)
    {
        return -x;
    }

    return x;
};

Console.WriteLine(absolutniHodnota(-5)); // 5
```

Delegát

Co je delegát

- **delegát** = typ, který reprezentuje odkaz na funkci
- říká, jak má funkce vypadat
- určuje:

- typ parametrů
- návratový typ
- umožňuje předávat funkce jako hodnoty

Příklad vlastního delegátu:

```
delegate int Operace(int a, int b);
```

Použití:

```
int Secti(int a, int b)
{
    return a + b;
}

Operace op = Secti;

Console.WriteLine(op(3, 5)); // 8
```

Func a Action

- V C# se často používají hotové delegáty:

Func

- používá se pro funkce, které vracejí hodnotu
- Func musí vždy něco vracet
- poslední typ je návratový

Příklad:

```
Func<int, int, int> secti = (a, b) => a + b;
```

- první int = typ prvního parametru
- druhý int = typ druhého parametru
- třetí int = návratový typ

Action

- používá se pro funkce, které nic nevracejí
- odpovídá metodám typu void

Příklad:

```
Action<string> vypis = text => Console.WriteLine(text);

vypis("Ahoj");
```

Funkce vyššího řádu

- **funkce vyššího řádu** přijímá funkci jako parametr
- nebo funkci vrací jako výsledek
- běžné ve funkcionálním programování
- často se používá u LINQ

Příklad:

```
List<int> cisla = new List<int> { -2, -1, 0, 1, 2, 3 };

var kladna = cisla.Where(x => x > 0);

foreach (int cislo in kladna)
{
    Console.WriteLine(cislo);
}
```

Vysvětlení:

- `Where` přijímá podmínku jako funkci
- `x => x > 0` je lambda funkce
- výsledkem jsou pouze kladná čísla

Closure

Co je closure

- **closure** = funkce si pamatuje proměnné ze svého okolí
- pamatuje si je i potom, co původní blok skončil
- používá se například u lambda funkcí
- umožňuje funkci pracovat s vnějším stavem

Příklad:

```
int nasobek = 2;

Func<int, int> vynasob = x => x * nasobek;

Console.WriteLine(vynasob(5)); // 10
```

- lambda používá proměnnou `nasobek`
- tato proměnná vznikla mimo lambda
- lambda ji má stále k dispozici

Praktické úlohy

1. Funkce pro součet

```
int Secti(int a, int b)
{
    return a + b;
}

Console.WriteLine(Secti(3, 5));
```

2. Procedura pro výpis

```
void Vypis(string text)
{
    Console.WriteLine(text);
}

Vypis("Ahoj");
```

3. Rozdíl mezi hodnotou a referencí

```
void ZvysHodnotou(int x)
{
    x++;
}

void ZvysReferenci(ref int x)
{
    x++;
}

int cislo = 5;

ZvysHodnotou(cislo);
Console.WriteLine(cislo); // 5

ZvysReferenci(ref cislo);
Console.WriteLine(cislo); // 6
```

4. Použití `out`

```
if (int.TryParse("123", out int cislo))
{
    Console.WriteLine("Převod se povedl: " + cislo);
}
else
{
    Console.WriteLine("Převod se nepovedl.");
}
```

5. Rekurzivní faktoriál

```
int Faktorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }

    return n * Faktorial(n - 1);
}

Console.WriteLine(Faktorial(5)); // 120
```

6. Lambda funkce v delegátu

```
Func<int, int> zdvojnásob = x => x * 2;

Console.WriteLine(zdvojnásob(6)); // 12
```

7. Lambda funkce v LINQ

```
List<int> cisla = new List<int> { 1, 2, 3, 4, 5, 6 };

var suda = cisla.Where(x => x % 2 == 0);

foreach (int cislo in suda)
{
    Console.WriteLine(cislo);
}
```